The University of Central Punjab

Faculty of Information Technology

Data Structures and Algorithms

Lab 2	
Topic	• Understanding Classes • Working on Three Different files • Templates • Growth rate(Time and Space Complexity)
Objective	The basic purpose of this lab is to revise some preliminary concepts of C++ that have been covered in the course of Introduction to Computing and Programming Fundamentals and Object-Oriented Programming.

M Ilyas L1F24BSAI0078 AD1

Instructions:

- Indent your code.
- Comment your code.
- Use meaningful variable names.
- Plan your code carefully on a piece of paper before you implement it.
- Name of the program should be the same as the task name. i.e. the first program should be Task_1.cpp
- **void main() is not allowed. Use int main()**
- **You have to work in multiple files. i.e separate .h and .cpp files**
- **You are not allowed to use system("pause")**
- **You are not allowed to use any built-in functions**
- **You are required to follow the naming conventions as follow:**
    - **Variables:** firstName; (no underscores allowed)
    - **Function:** getName(); (no underscores allowed)
    - **ClassName:** BankAccount (no underscores allowed)

Students are required to complete the following tasks in lab timings.

Task 01:

• *Create a function template to find the maximum of two values.*

Instructions:

1. Define a function template findMax that accepts two arguments of any type.

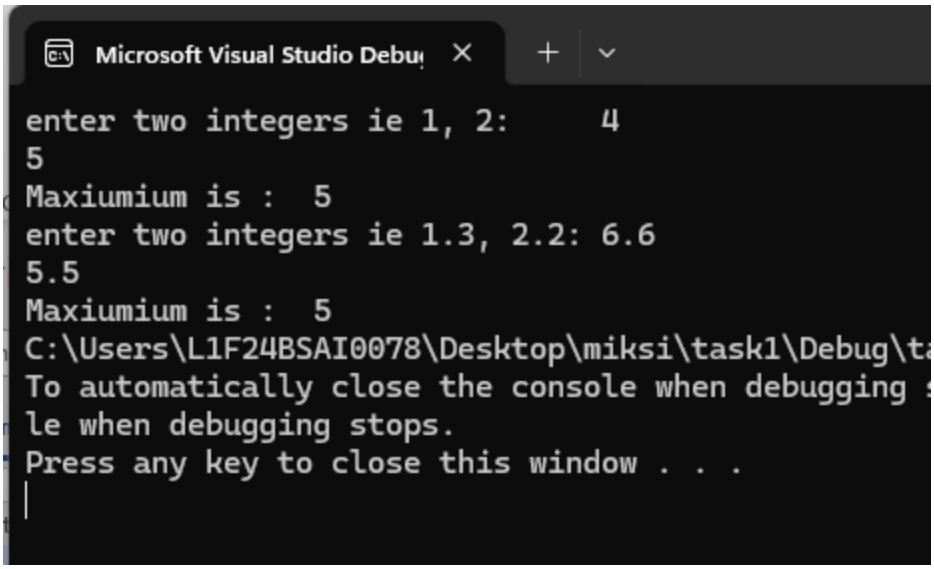
2. Compare the two values and return the maximum.

3. Test the function with integers and floating-point numbers.

source code:

```
// task1.cpp : This file contains the 'main' function. Program execution begins and ends
there.
//

#include <iostream>
using namespace std;
template <class T>
T findMaxi(T a, T b)
{
    if (a > b) {
        return a;
    }
    else {
        return b;
    }
}
int main()
{
    int num_1, num_b;
    cout << "enter two integers ie 1, 2:\t";
    cin >> num_1 >> num_b;
    cout<<"Maxiumium is :\t"<<findMaxi(num_1, num_b);

    float num_a, num_c;
    cout << "\nenter two integers ie 1.3, 2.2:\t";
    cin >> num_a >> num_c;
    cout << "Maxiumium is :\t" << findMaxi(num_1, num_b);

    return 0;
}
```

Output:



Max of 10 and 20 is: 20

Max of 5.5 and 2.3 is: 5.5

Task 02:

• *Create a class template to implement a simple array that can store elements of any data type.*

Instructions:

1. Define a class template Array with a constructor that takes the size of the array.

2. Create functions to set and get elements at a given index.

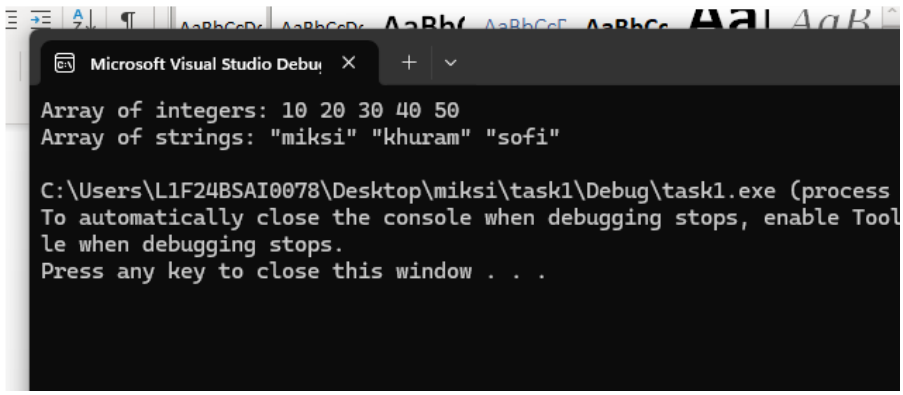
3. Test the class with integers and strings.

Source code :

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;
template <typename T>
class Array {
    T* arr;
    int size;

public:
    Array(int siz) {
        size = siz;
        arr = new T[size];
    }

    void setter(int indx, T value) {
        if (indx >= 0 && indx < size) {
            arr[indx] = value;
        }
        else {
            throw out_of_range("Index out of range");
```

```
        }
    }

    T getter(int indx) {
        if (indx >= 0 && indx < size) {
            return arr[indx];
        }
        throw out_of_range("index is out out of range");
    }

    void show() {
        for (int i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    ~Array() {
        delete[] arr;
    }
};

int main() {
    // for integers
    Array<int> intArr(5);
    intArr.setter(0, 10);
    intArr.setter(1, 20);
    intArr.setter(2, 30);
    intArr.setter(3, 40);
    intArr.setter(4, 50);

    cout << "Array of integers: ";
    intArr.show();

    // for strings
    Array<string> strArr(3);
    strArr.setter(0, "\"miksi\"");
    strArr.setter(1, "\"khuram\"");
    strArr.setter(2, "\"sofi\"");

    cout << "Array of strings: ";
    strArr.show();

    return 0;
}
```

Output:



Array of integers: 10 20 30 40 50

Array of strings: "apple" "banana" "cherry"

Task 03:

• Write a function template to calculate the sum of an array.

Instructions:

1. Define a function template arraySum that takes an array and its size as arguments.

2. The function should return the sum of all elements in the array.

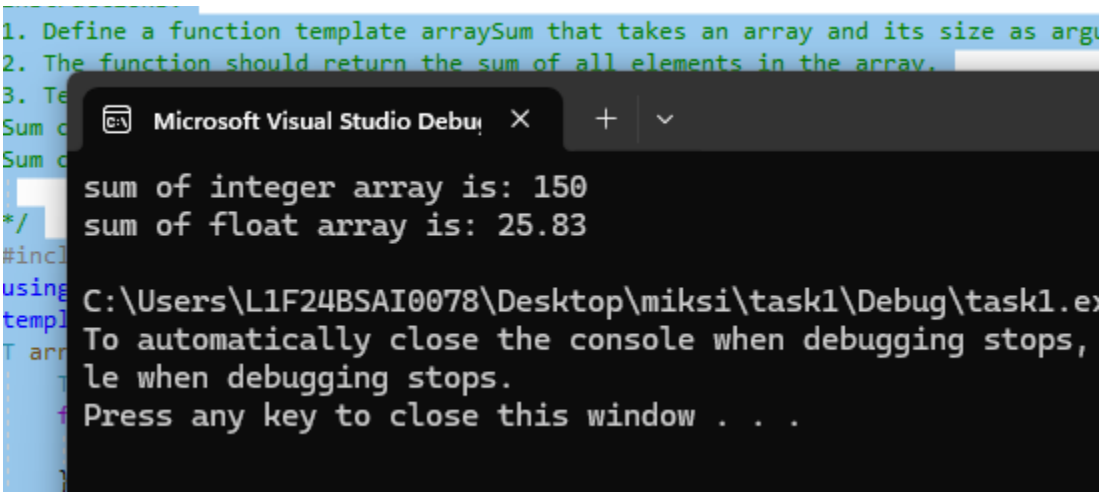
3. Test the function with both integer and float arrays.

Source code :

```
/*
Task 03:
• Write a function template to calculate the sum of an array.
Instructions:
1. Define a function template arraySum that takes an array and its size as arguments.
2. The function should return the sum of all elements in the array.
3. Test the function with both integer and float arrays.
Sum of integer array: 150
Sum of float array: 25.75

*/
#include <iostream>
using namespace std;
template <typename T>
T arraySumm(T arr[], int size) {
    T sum = T(0);
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return sum;
}

int main() {
    int int_arr[] = { 10, 20, 30, 40, 50 };
    int intSize = sizeof(int_arr) / sizeof(int_arr[0]);  //for actual size colculation

    cout << "sum of integer array is: " << arraySumm(int_arr, intSize) << endl;
```

```
    float float_arr[] = { 10.5, 5.1, 10.23 };
    int floatSize = sizeof(float_arr) / sizeof(float_arr[0]);

    cout << "sum of float array is: " << arraySumm(float_arr, floatSize) << endl;

    return 0;
}
```

Output:



Sum of integer array: 150

Sum of float array: 25.75

Task 4 (Class Participation Task)

Compute the Time and space complexity of the following codes, according to their line by line execution. Finally compute their big(O) complexity.

a.

```
void phytagorean(int value)
{
  for(int i = 1; i <= value; i++)
  {
        for(int j = 1; j <= value; j++)
        {
              for(int k = 1; k <= value; k++)
              {
              int num1 = (i*i) + (j*j);
```

```
                int num2 = (k*k);
                if (num1 == num2)
                        cout<<"Pair is: (" << i << ", " << j << ", " << k << ")" << endl;
                }
            }
        }
    }
```

Answer:

Line of Code	Cost	Frequency (Executions)
`void phytagorean(int value) {`	**c_0**	**1**
`for(int i = 1; i <= value; i++) {`	**c_1**	**n + 1**
`for(int j = 1; j <= value; j++) {`	**c_2**	**n(n + 1)**
`for(int k = 1; k <= value; k++) {`	**c_3**	**n^2(n + 1)**
`int num1 = (i*i) + (j*j);`	**c_4**	**n^3**
`int num2 = (k*k);`	**c_5**	**n^3**
`if (num1 == num2)`	**c_6**	**n^3**
`cout << "Pair is: ...`	**c_7**	**n^3(Worst Case)**

b.

```
void RangeCheck(int arr[],int sixe, int num1, int num2){
    int counter = 0;
    for (int i=0; i<sixe; i++)
    {
        if (arr[i] >= num1 && arr[i] <= num2)
        {
            counter++;
            cout << arr[i] << "is in the range " << endl;
```

```
        }
    }
    cout << "Total Numbers in range are: " << counter << endl;
}
```

Answer:

Line of Code	Cost	Frequency (Executions)
`void RangeCheck(int arr[], int sixe...) {`	**c_0**	**1**
`int counter = 0;`	**c_1**	**1**
`for (int i=0; i<sixe; i++) {`	**c_2**	**n + 1**
`if (arr[i] >= num1 && arr[i] <= num2)`	**c_3**	**n**
`counter++;`	**c_4**	**n(Worst Case)**
`cout << arr[i] ...`	**c_5**	**n (Worst Case)**
`cout << "Total Numbers ..."`	**c_6**	**1**

C .

```
int main() {
int n;

cout << "Enter size: ";
cin >> n;

int arr[n];

cout << "Enter elements: ";
for(int i = 0; i < n; i++) {
cin >> arr[i];
}

for(int i = 0; i < n-1; i++) {
for(int j = 0; j < n-i-1; j++) {
if(arr[j] > arr[j+1]) {
```

```
int temp = arr[j];
arr[j] = arr[j+1];
arr[j+1] = temp;
}
}
}

cout << "Sorted array: ";
for(int i = 0; i < n; i++) {
cout << arr[i] << " ";
}

return 0;
}
```

Answer:

Line of Code	**Cost**	**Frequency (Executions)**
`int n; cin >> n; int arr[n];`	**c_1**	**1**
`for(int i = 0; i < n; i++)`	**c_2**	**n + 1**
`cin >> arr[i];`	**c_3**	**n**
`for(int i = 0; i < n-1; i++)`	**c_4**	**n**
`for(int j = 0; j < n-i-1; j++)`	**c_5**	
`if(arr[j] > arr[j+1])`	**c_6**	**n\(n-1)}{2}**
`int temp = arr[j]; ...` **(Swaps)**	**c_7**	**{n\n-1)}{2}(Worst Case)**
`for(int i = 0; i < n; i++) cout ...`	**c_8**	**n**